



Modelo Computacional

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

12 de marzo de 2026

1. Introducción	1
1.1. Dominio	2
1.2. Especificidad del Dominio	2
2. Artefactos	3
3. Sistema de Tipos	5
3.1. Tipado	5
3.2. Mapeo del Dominio	6
4. Construcciones	7
4.1. Estructuras de Control	7
4.2. I/O	8
4.3. Glosario	8

1. Introducción

Antes de comenzar a desarrollar un lenguaje se debe estar en presencia de una *necesidad arquitectural*, para la cual el lenguaje y su compilador representan una solución adecuada.

La necesidad arquitectural debe involucrar un *dominio* del mundo real, el cual debe ser abstraído para dar lugar a un *modelo computacional*. Este modelo computacional es el que finalmente será manipulado programáticamente a través del lenguaje creado. Se espera que dicha manipulación programática del dominio abstraído permita escalar sistemáticamente la transformación concreta y real del mismo, es decir, que fomente un alto grado de automatización, de integración, de mejora continua, permitiendo a su vez la aplicación de tecnologías propias del desarrollo de software (i.e., versionado, testing, modularización, reutilización, IDEs, etc.).

1.1. Dominio

Un proveedor de infraestructura en la nube (IaaS; *Infrastructure as a Service*), necesita desplegar diferentes componentes de red (routers, switches, firewalls, proxies, instancias, máquinas virtuales, dispositivos de almacenamiento, etc.).

Actualmente, dichos recursos se despliegan de forma manual, es decir, un cliente determinado contacta al equipo de ingenieros del proveedor indicando explícitamente la lista de recursos, sus características de capacidad y los diagramas de conexión de la arquitectura de red que necesitan. Luego, el equipo de ingenieros implementa la solución y al finalizar otorga accesos al equipo de desarrollo del cliente, para que puedan desplegar sus proyectos dentro de dicho ecosistema.

Este mecanismo manual de interacción con el cliente y despliegue de la solución es factible en un principio, pero el proveedor de infraestructura se encuentra frente a un aumento de demanda significativa debido a que múltiples clientes se aproximan con diferentes necesidades de capacidad, en particular porque el proveedor cada vez adquiere mayor reputación en el mercado.

En este instante surge una **necesidad arquitectural**: el proveedor necesita que el cliente (o en su defecto, el equipo de ingenieros), pueda especificar su arquitectura de forma programática, y que esta especificación despliegue automáticamente los recursos sobre una red con capacidad suficiente y de forma genérica.

Debido a que la necesidad arquitectural implica un **escalamiento en el desarrollo** (en este caso, tanto de *hardware* como de *software*), la solución es programática, y el dominio es específico (arquitectura de redes, sistemas distribuidos y *data centers*), la construcción de un lenguaje y su compilador deberían solventar la necesidad.

El dominio en definitiva implica determinar dentro del universo posible, todo aquello que es relevante directa o indirectamente para lograr un objetivo final, sea este construir una solución, o entender, manipular, simular y emular un fenómeno del mundo real.

1.2. Especificidad del Dominio

Un dominio puede ser al menos de uno de dos tipos diferentes:

- (I). **General**: El dominio en realidad abarca múltiples dominios en simultáneo (por ejemplo, el dominio del aprendizaje automático y el dominio de los sistemas multiagente), o incluso todos ellos dentro de cierta categorización (por ejemplo, el dominio de las ciencias de la vida).
- (II). **Específico**: En este caso el dominio está concentrado en un tema o grupo de entidades en específico, donde las relaciones entre ellos son suficientemente fuertes (hay alta cohesión entre ellas), y además conocidas con el detalle suficiente.

Para estas especificidades del dominio, se presentan entonces sus contrapartes en el mundo de los lenguajes formales:

- (III). **GPL (General Purpose Language):** Son aquellos lenguajes que no fueron diseñados con un dominio objetivo específico en particular y que por lo tanto permiten manipular diferentes dominios y crear diferentes aplicaciones según sea necesario, incluso utilizando tecnologías y algoritmos diversos. Ejemplos de estos lenguajes son C, C++, C#, Java, Haskell, Python, Ruby, TypeScript, entre muchos otros.
- (IV). **DSL (Domain Specific Language):** Lenguajes que se especifican en un único dominio en particular y que por lo tanto poseen una sintaxis que facilita la manipulación directa de ciertos conceptos del mismo, incrementando el nivel de abstracción disponible, ocultando ciertos detalles de implementación. Ejemplos válidos de DSL son Alloy, Bash, CSS, Cypher, EPICS, HTML, JML, JSON, MATLAB, Nginx, SQL, Terraform, UML, Verilog, XML, etc.

El uso de un DSL para desarrollar cierta solución permite que el programador se concentre específicamente en la interacción y manipulación de componentes de alto nivel, en lugar de los detalles propios de la implementación subyacente o de la mecánica que existe entre dichos componentes.

Por ejemplo, para el dominio de la **Sección § 1.1: Dominio**, un DSL permitiría manipular directamente los recursos necesarios de forma programática (*routers, switches, firewalls, discos, etc.*), pero sin entrar en detalle sobre el sistema de despliegue de los cambios y configuración de los mismos, de la mecánica de aprovisionamiento de recursos, o del orden en el cual dichos componentes se deben crear e interconectar. El resultado es que el equipo de desarrollo puede encargarse de programar la solución deseada, libre de toda interferencia. Por otro lado, el constructor de la solución (el DSL y su compilador), se puede encargar de optimizar los detalles de implementación, aprovisionamiento y configuración de los recursos sin afectar a los clientes.

Es posible sobre un DSL o GPL introducir mejoras dentro del compilador sin afectar la sintaxis de los mismos, debido a que el lenguaje en sí mismo actúa como una interfaz de programación (i.e., una API, aunque no necesariamente “de aplicación”), entre el dominio que se intenta manipular (en el ejemplo, un *data center*, sus redes y recursos) y la descripción de dicha manipulación (el programa o especificación).

Por lo tanto, un lenguaje permite **desacoplar** el dominio, de la mecánica del modelo computacional que lo abstrae. En este sentido: Bash permite interactuar entre diferentes procesos independientes y el OS subyacente uniformemente (aunque crípticamente); Cypher permite consultar, actualizar y generar grafos y subgrafos en una base de datos; EPICS permite controlar el hardware y los mecanismos operacionales detrás de aceleradores y colisionadores de partículas; Nginx simplifica la creación y configuración de servidores, balanceadores de carga y proxies; JSON y XML simplifican el transporte de datos en un sistema distribuido; y MATLAB integra la construcción y ejecución de algoritmos numéricos y simulaciones.

2. Artefactos

Es relevante determinar qué clase de *artefactos* de entrada y salida van a intervenir en la construcción de un lenguaje y su compilador.

El esquema clásico implica que el artefacto de entrada es un programa escrito en el lenguaje que el compilador desarrollado acepta (sea GPL o DSL), es decir, el programa es una secuencia de símbolos o cadena de caracteres, usualmente del gran alfabeto **Unicode**. En su salida, se espera un único artefacto: un *ejecutable* (DLL, EXE, JAR, SO, etc.).

Sin embargo, un compilador es, en definitiva, un mecanismo abstracto de transformación de símbolos, con lo cual la entrada bien puede ser uno o varios documentos en binario u otra codificación no-textual (*e.g.*, imágenes, sonidos, etc.), y su salida no necesariamente debe ser única (*i.e.*, un único archivo), ni ejecutable (de la misma forma, puede ser una imagen, un sonido u otro programa codificado en otro lenguaje más o menos abstracto, ver **Fig. (2.1)**).

Para el dominio ejemplificado en las secciones anteriores, el programa de entrada podría ser una especificación textual de los recursos requeridos y de su configuración y capacidad requerida, junto con una relación de dependencias que indique el nivel de composicionalidad y jerarquización que existe entre ellos.

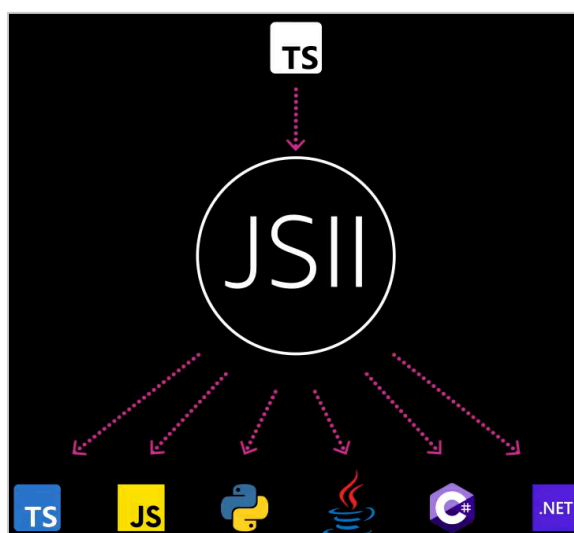


Figura 2.1: JSII es un compilador polígloa que permite traducir un programa escrito en TypeScript hacia JavaScript, Python, Java, C# o .NET.

¿Cuál debería ser el artefacto de salida en este caso? Debería ser un conjunto de operaciones que luego de ejecutarse efectivamente desplieguen y configuren en el orden requerido todos esos componentes y recursos, respetando las dependencias entre ellos. Esto involucra diversas tecnologías (*e.g.*, un *router* no se configura igual que un *switch*), con lo cual este conjunto de operaciones bien puede consumir múltiples interfaces (todas ellas externas al compilador, obviamente).

En definitiva, todos los detalles quedan ocultos detrás de un lenguaje simplificado y de alto nivel. La complejidad final queda del lado del proveedor, quien conoce perfectamente el dominio en cuestión, su mecánica interna y sus limitaciones. Ahora, el proveedor sabe que todas las especificaciones que recibe se presentan de la misma manera (mediante el DSL), y por ende puede automatizar (y escalar) su procesamiento.

3. Sistema de Tipos

Un *sistema de tipos* se compone tanto de los tipos disponibles para crear objetos, variables, constantes y entidades dentro de un lenguaje, así como también de las reglas que permiten manipular o inferir el tipo para una expresión dada.

En el contexto de un lenguaje y desde un punto de vista puramente matemático, un *tipo de dato* no es nada más que un conjunto de elementos (que puede ser finito o infinito), y las operaciones definidas sobre estos conjuntos son por lo tanto *operadores*, es decir, funciones matemáticas unarias, binarias, ternarias, etc., que trabajan sobre algún grupo cartesiano de estos conjuntos.

De esta manera, es común representar conjuntos finitos mediante tipos enumerados en Java o TypeScript (usando `enum`), y conjuntos infinitos mediante otros tipos de datos numéricos (como `int`, o `double`), o de longitud arbitraria (*arrays* y *strings*). Nótese que en realidad, algunos lenguajes como C, C++ o Java proporcionan un espacio máximo y fijo predefinido para ciertos tipos de datos que representan cantidades o conjuntos infinitos, con lo cual esta discretización produce, en definitiva, otro conjunto finito de valores posibles (e.g., `double` posee un límite máximo y una precisión definida, claramente no puede representar la totalidad de números reales).

Según el tipo de lenguaje y su nivel de abstracción, estos tipos disponibles serán de mayor o menor **orden**, por ejemplo, algunos permiten la definición de *funciones*, pero no permiten el uso de funciones como valor (recordar por ejemplo, el uso de punteros a función en C, el uso de funciones anónimas en Java, o las *arrow-functions* en JavaScript), y por ende, no se pueden redactar algoritmos que manipulen, produzcan o utilicen funciones de alto orden.

Finalmente, un lenguaje debe poder **escalar en complejidad**, con lo cual, además de proveer un conjunto básico de tipos de datos, se debe proveer un mecanismo de generación de nuevos tipos definidos por el programador. La definición de funciones, estructuras, enumerados, clases, prototipos, interfaces, y genéricos, entre muchos otros, son algunas de las formas que tienen los lenguajes de facilitar el armado de conceptos y abstracciones de alto orden.

3.1. Tipado

En general, los lenguajes se subdividen en dos grandes grupos:

- (I). Aquellos que poseen tipado **estático**, donde la asignación y validación de tipos se realiza durante la fase de compilación del lenguaje y por lo tanto se garantiza que la solución final (los artefactos de salida), están bien contruidos desde este punto de vista.
- (II). Aquellos que poseen tipado **dinámico**, donde las reglas del sistema de tipos se aplican en tiempo de ejecución (*runtime*), y por lo tanto el artefacto se debe ejecutar o desplegar (o la operación que sea apropiada sobre él), para determinar si efectivamente las operaciones que se llevaba cabo sobre él son válidas o no (en cuyo caso el programa original o artefacto de entrada era inválido en primer lugar).

Los lenguajes con tipado estático ofrecen mayores garantías y por lo tanto una reducción significativa de errores y una minimización del riesgo inducido al ejecutar o desplegar la solución. El tipado *dinámico*, por el contrario, pospone dichas garantías hacia un tiempo de ejecución donde ya no hay vuelta atrás; el artefacto funcionará o no según un programa que ya fue compilado y que no se encuentra disponible para modificación.

Más aún, algunos lenguajes de la categoría dinámica promueven dos formas básicas de asignar un tipo sobre una variable o expresión: aquellos que respetan el primer tipo de dato asignado a la variable durante su ciclo de vida (**strong typing**), y aquellos que permiten modificar el mismo arbitrariamente durante su procesamiento (**weak typing**).

Por último existe una variante adicional denominada *duck-typing*, bajo la cual un objeto o variable se asume de cierto tipo si posee los métodos y propiedades que dicho tipo define, y de esta manera un objeto podría pertenecer en simultáneo a múltiples tipos de datos, análogo (pero no equivalente), al efecto que las *interfaces* tienen sobre los lenguajes orientados a objetos como Java o C++.

3.2. Mapeo del Dominio

Dado un dominio, es necesario mapear los conceptos y entidades que este mismo define hacia el *modelo computacional*, es decir, a la sintaxis que finalmente hará las veces de abstracción dentro del lenguaje. En este sentido, el diseñador del lenguaje debería definir un sistema de tipos que represente a aquellos elementos fundamentales del dominio con los cuales construir las interacciones y algoritmos que llevarán a la obtención de una solución.

Para el dominio ejemplificado, es fácil identificar ciertos recursos o **sustantivos** que bien podrían mapearse a tipos de datos. Segregando el grupo de conceptos en capas (red, almacenamiento y cómputo), sería posible definir como mínimo, por ejemplo:

- (I). **Red:** Address, gateway, network, packet, route.
- (II). **Almacenamiento:** Volume.
- (III). **Cómputo:** Instance.

Estos conceptos bien pueden mapearse a tipos de datos del dominio, más allá de que también coexistan con tipos clásicos, como *string*, *boolean* e *integer*. Estos tipos permiten acercar aún más al desarrollador hacia el modelo mental real que posee del dominio y de aquello que quiere construir y especificar (la arquitectura de red).

El agregado de expresividad que aportan estos nuevos tipos de dato permite también definir nuevos **verbos** o acciones sobre ellos (i.e., operadores), como por ejemplo *attach* (asociar un volumen de almacenamiento a una instancia), así como también indicadores relacionales o composicionales como *in* (una preposición que indicaría que cierta instancia se encuentra *dentro de/en* cierta red).

Se definen de esta manera *dependencias adicionales*, por ejemplo, definir que una instancia debe pertenecer a cierta red mediante *in*, es un indicador para el compilador de que efectivamente primero se debe crear la red antes de asignar una instancia en ella (y que de igual forma, la instancia se debe crear antes de intentar su introducción en una red existente). Esto guía naturalmente la forma en la que el compilador efectuará la emisión final de sus artefactos, en este caso, de las operaciones que debe ejecutar sobre la infraestructura.

4. Construcciones

La sintaxis de un lenguaje se compone de diferentes *construcciones*, es decir, esquemas que permiten definir ciertos aspectos del comportamiento del programa durante su compilación y ejecución.

4.1. Estructuras de Control

Estas construcciones permiten manipular el flujo de ejecución normal de un programa. Hay al menos dos tipos, **condicionales** e **iterativos**.

El propósito de las estructuras de control *condicionales* es el de decidir qué fragmento del programa ejecutar o habilitar dentro o fuera del compilador en un momento determinado. Ejemplos clásicos son las estructuras IF-THEN-ELSE o incluso los bloques SWITCH. En algunos lenguajes como Haskell, la estructura debe manipular la totalidad del universo sobre el que actúa, por ejemplo, para una condición dentro de un IF, debe existir tanto un bloque definido para THEN como para ELSE; sin embargo, para lenguajes como C o TypeScript no es necesario definir lo que sucede dentro del bloque ELSE.

Una forma más avanzada de condicional se denomina *pattern matching*, el cual opera similar a un esquema con SWITCH, pero con dos ventajas adicionales: (I) el conjunto de patrones debe cubrir la totalidad del tipo de dato (no puede fallar); (II) permite expresar patrones explícitos o más complejos que un SWITCH clásico (mejora de la DX). Un ejemplo de *pattern matching* en Haskell, para definir la función *factorial*:

```
factorial :: Int -> Int
factorial k =
  case k of
    0 -> 1
    _ -> k * factorial (k - 1)
```

Nótese que “_” implica cualquier otro entero que sea distinto de 0 (cero).

Las estructuras de control *iterativo*, por otro lado, facilitan la escalabilidad y reusabilidad de las operaciones definidas en el programa, ya que es deseable en general, manipular grandes cantidades de datos de forma simplificada, uno de los principales objetivos de casi cualquier aplicación (i.e., la automatización de tareas repetitivas).

Ejemplos típicos son el bucle WHILE, DO-WHILE, FOR y RANGE-BASED-FOR. Estos bucles representan un tipo de iteración denominada **externa** (o explícita), donde el objeto que está en proceso se expone junto con el mismo mecanismo de acceso al mismo (por ejemplo, un índice sobre un *array* o vector). Sin embargo, los mecanismos de iteración **interna** (o implícita), ocultan el acceso al objeto (aunque no el objeto en sí), con lo cual el esquema es de tipo *declarativo* en lugar de *imperativo*¹, como es el caso de la iteración externa.

¹ Una construcción *declarativa* (o lenguaje, si aplica a todas las construcciones que posee), es aquella que especifica directamente lo que se desea obtener (su forma o valor), pero que no especifica la manera o secuencia de operaciones que conllevan a la obtención de dicha forma o valor. Si la secuencia es explícita, entonces se trata de una construcción *imperativa*.

4.2. I/O

Muchos lenguajes requieren ciertos puntos de contacto entre el espacio sobre el que definen sus componentes y el flujo de su ejecución, y el entorno en el que se encuentran. Es factible que un programa requiera en primer lugar de un *entry-point* (punto de entrada), que representa aquel lugar dónde comienza su ejecución o la declaración de sus componentes (función `main` en C o Java).

En un ejecutable, este punto de entrada se corresponde literalmente con el inicio de su ejecución, pero también existen artefactos que poseen múltiples *entry-points* (tales como una DLL; *Dynamic-Link Library*).

Por otro lado, un punto de contacto entre el posible ejecutable está definido por sus entradas y salidas (no confundir estas entradas y salidas del artefacto, con los artefactos de entrada y salida del compilador). En cualquier ejecutable naturalmente existen dos puntos bien definidos: STDIN y STDOUT, con lo cual es común, pero no necesario en lo absoluto, que un programa ofrezca una manera de introducir datos desde el exterior, y de forma análoga exteriorizarlos.

Estos mecanismos de entrada y salida pueden ser útiles para aplicar algún tipo de coordinación entre diferentes artefactos, o simplemente para visualizar ciertos estados del programa durante su ejecución o compilación, pero nótese que es posible que dicho mecanismo esté naturalmente **abstraído** dentro del *modelo computacional* que impone el lenguaje, y que si bien el mismo puede existir, no necesariamente se debe ofrecer alguna construcción explícita para que el programador la pueda editar o afectar directamente.

Continuando el ejemplo, está claro que desplegar un *router* en la periferia de una red implica configurarlo, sin embargo, el mecanismo de acceso a dicho dispositivo y los detalles de su configuración podrían estar ocultos y ser generados y gestionados automáticamente como parte de la compilación. El usuario no tendría que preocuparse de este mecanismo ni del transporte de ciertos parámetros entre ambas partes, de ninguna manera. El punto de contacto existe, pero es virtualmente inaccesible para el programador.

4.3. Glosario

Se definen a continuación algunos nombres que comúnmente se utilizan para definir ciertas partes o construcciones en varios lenguajes, a modo de estandarizar de alguna manera el esquema de nombramientos utilizados (esto es particularmente importante a la hora de definir los símbolos *no-terminales* de una gramática):

Construcción	Definición
Argumento	El valor concreto que posee un parámetro en el momento de ejecutar o llamar una función.
Asignación	Análogo a la definición, pero en este caso no necesariamente forma parte de la inicialización de la variable o función en cuestión, bien podría ser una segunda asignación de variable.
Bloque	Una sección de código que está relacionada bajo cierto <i>scope</i> (alcance),

	dentro del cual existen ciertas variables, funciones o nombres disponibles. Es común indicarlos entre {}, pero no necesariamente (Python usa <i>indentación</i> para determinar dónde abre y cierra un bloque).
Comentario	Sección de documentación dentro de un programa, que usualmente es ignorada por el compilador antes de comenzar el análisis sintáctico.
Condición	Una expresión que es de tipo <i>booleana</i> .
Constante	Variable cuyo valor se mantiene constante una vez definido o inicializado, (valga la contradicción en su nombre).
Declaración	El anuncio de una nueva variable o función, pero sin indicar su contenido o valor. Por ejemplo, int x ; es una declaración de un entero, pero nótese que no se le asigna ningún valor.
Definición	La especificación concreta del valor que tiene una variable o función, tal como x = 0 ; . La definición puede ocurrir en el mismo lugar de su declaración, o lejos de ella.
Expresión	Una construcción para la cual se le puede asignar, computar o inferir un tipo de dato determinado. Construcciones sin tipo, no se consideran expresiones.
Factor	Análogo al término, pero en este caso la precedencia es superior o máxima.
Función	Un fragmento de código con o sin parámetros, con tipo de retorno, referenciable y ejecutable bajo cierto identificador.
Función Anónima	Función que no posee nombre y que por lo tanto se asigna a alguna variable o se ejecuta <i>inline</i> dentro de alguna expresión (u otra función).
Identificador	El nombre que determina la manera de referenciar una variable, clase, función, etc., dentro del programa. En general deben ser únicos dentro del mismo <i>scope</i> .
Inicializador	Construcción que se ejecuta al inicio de un bucle, o que define un array o vector al momento de su declaración.
Interfaz	Clase que posee solo métodos públicos sin definición.
Keyword	Una palabra clave reservada del lenguaje. Se espera que ningún identificador de variable utilice palabras de este tipo.
Literal	Un valor determinado, expresado literalmente, como por ejemplo true , 3.1415 u "hola mundo" .
Método	Función o procedimiento que pertenece a una clase o interfaz.
Modificador	Keyword que permite alterar el ciclo de vida de alguna variable, función o construcción en general, como por ejemplo static , sealed o const .
Modificador de Acceso	Análogo a modificador, pero en este caso afecta específicamente al grupo de <i>scopes</i> que están habilitados a acceder a cierta variable, función o construcción.
Operador	Función que tiene cierta aridad y se define sobre los tipos fundamentales que provee el lenguaje.

Parámetro	El nombre o identificador de una de las variables que recibe una función para operar.
Procedimiento	Una función que en realidad no lo es desde un punto de vista matemático, debido a que no retorna nada (tal como las funciones que retornan void en C o C++).
Propiedad	Una variable que pertenece a una clase o estructura de datos.
Sentence	Una construcción similar a un <i>statement</i> , pero que cabe en una línea o expresión por sí sola y no contiene ningún bloque en su interior.
Signature	El tipo de dato de una función, lo cual incluye su tipo de retorno y el tipo de todos sus argumentos.
Statement	Una construcción que efectúa parte de un programa, como por ejemplo el conjunto IF-THEN-ELSE.
Término	Una subdivisión principal dentro de una expresión, usualmente la de menor precedencia.